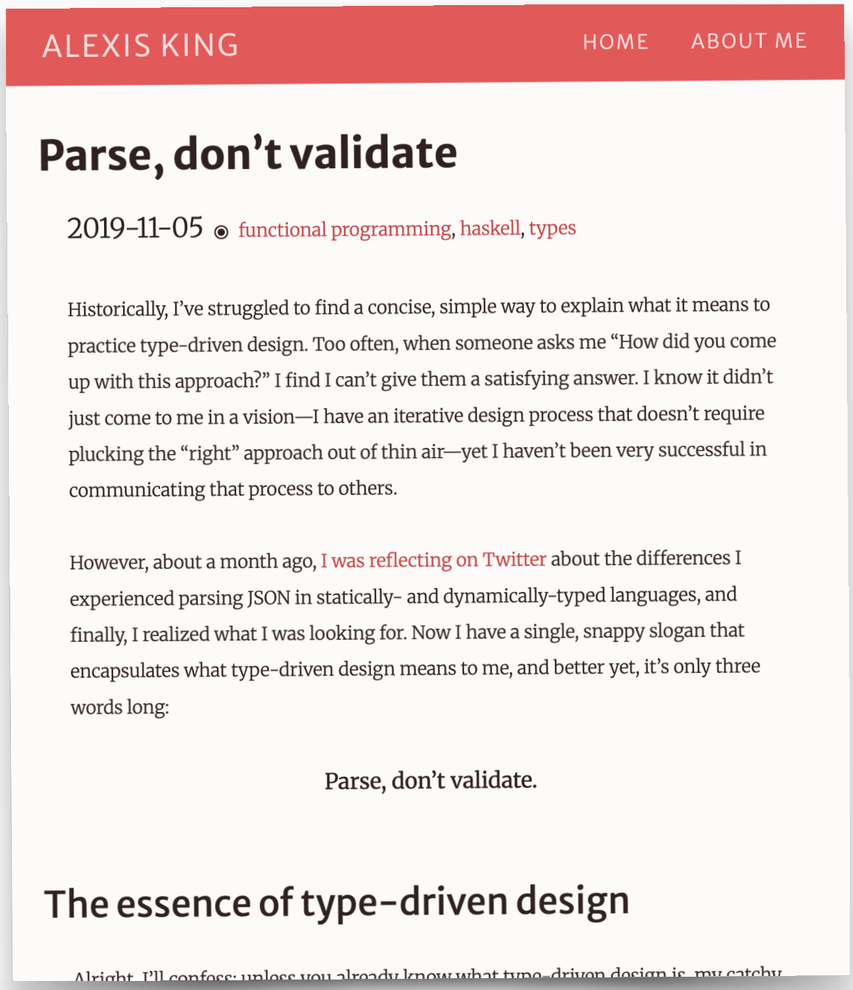
The unreasonable effectiveness of CONSTRUCTIVE DATA MODELING

Alexis King

Software Should Work

ABOUT ME



ALEXIS KING HOME ABOUT ME

Parse, don't validate

2019-11-05 ◉ functional programming, haskell, types

Historically, I've struggled to find a concise, simple way to explain what it means to practice type-driven design. Too often, when someone asks me "How did you come up with this approach?" I find I can't give them a satisfying answer. I know it didn't just come to me in a vision—I have an iterative design process that doesn't require plucking the "right" approach out of thin air—yet I haven't been very successful in communicating that process to others.

However, about a month ago, I was reflecting on Twitter about the differences I experienced parsing JSON in statically- and dynamically-typed languages, and finally, I realized what I was looking for. Now I have a single, snappy slogan that encapsulates what type-driven design means to me, and better yet, it's only three words long:

Parse, don't validate.

The essence of type-driven design

Alright, I'll confess: unless you already know what type-driven design is, my catchy

PREVIOUSLY

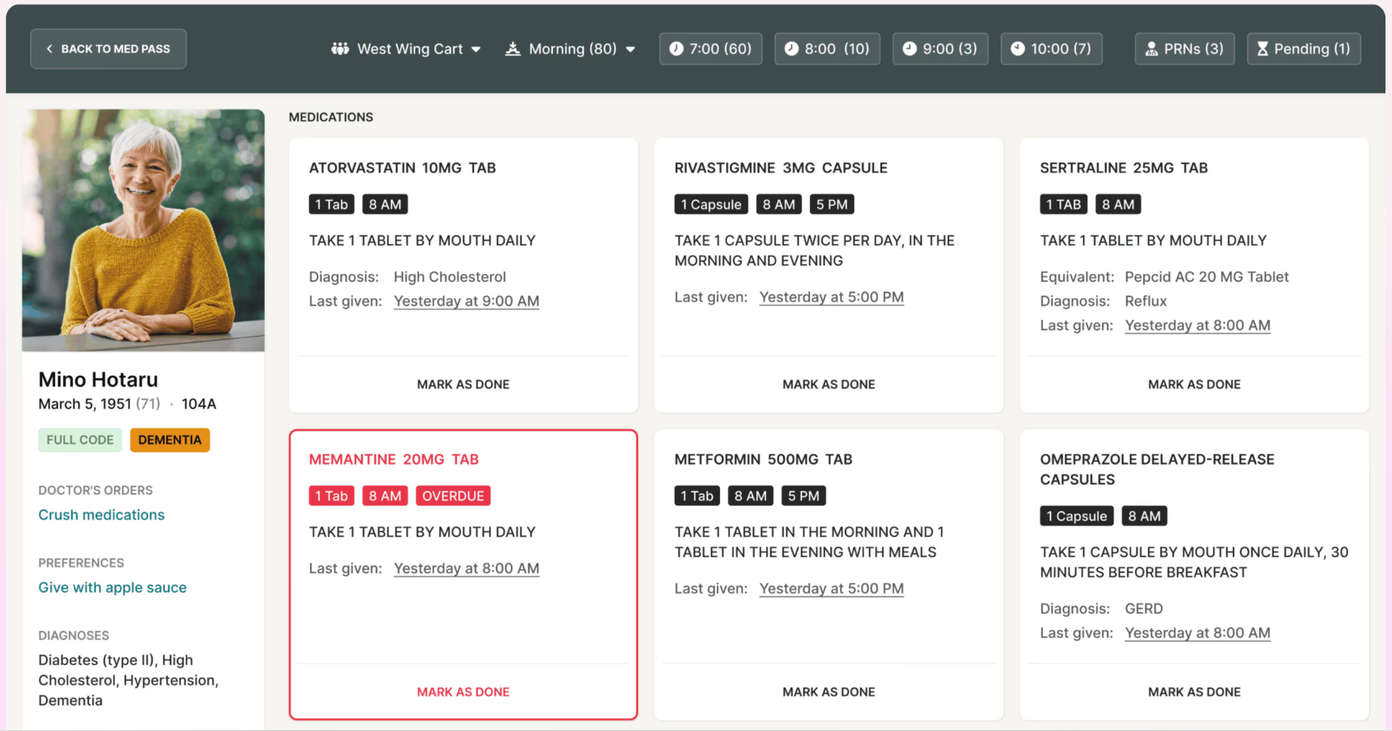
Haskell /GHC

NOW

Scala @ August

August Health eMAR

Our smarter, fully-integrated eMAR dramatically improves your medication management and clinical operations.



Software should work, indeed!

STATIC TYPING

15 years ago: the static typing dark ages.



Java

→ infamously conservative

→ deeply entangled with OOP



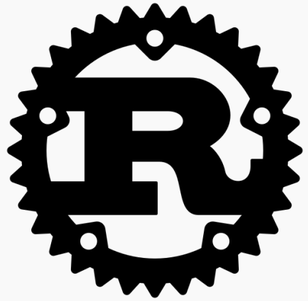
C++

→ extremely complicated

→ very unsound

Static typing: old and crusty!

THE RENAISSANCE



Rust

- → focus on safety/soundness
- → sum types, pattern-matching
- → traits and associated types



TypeScript

- → structural typing
- → fancy type inference
- → type-level programming

Static typing is cool again!

BUT AT WHAT COST?

Extreme increase in type system complexity!

→ Programmers don't know the whole type system
→ Typechecker treated as a black box
→ Time spent "fighting with the typechecker"
→ Ever-growing set of type system features

What made programmers willing to accept this?

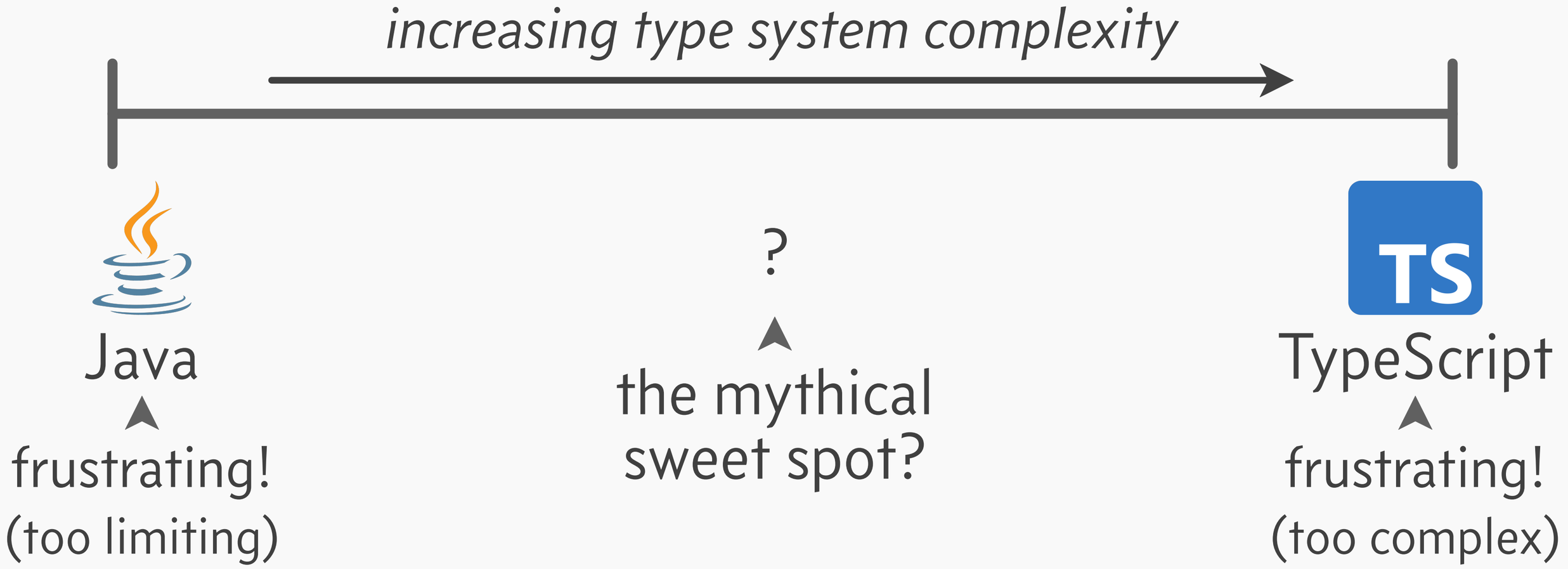
GRADUAL TYPING

- → TypeScript is supposed to be “JavaScript, but typed”
- → Static semantics must be more complex to accommodate dynamic typing idioms

```
if (typeof x === "string") {
  useString(x)
}
```

- → Programmers have mental model of *dynamic* semantics
- → Type checker is perceived as a magic oracle

This makes me sad. :(



What if I told you…

→ …I don't ever feel like I'm fighting the typechecker.

→ …I feel equipped to use my type system to capture invariants without built-in language support.

→ …it involves *less* sophisticated type system machinery.

→ …you can do it, too, in the language you already use!

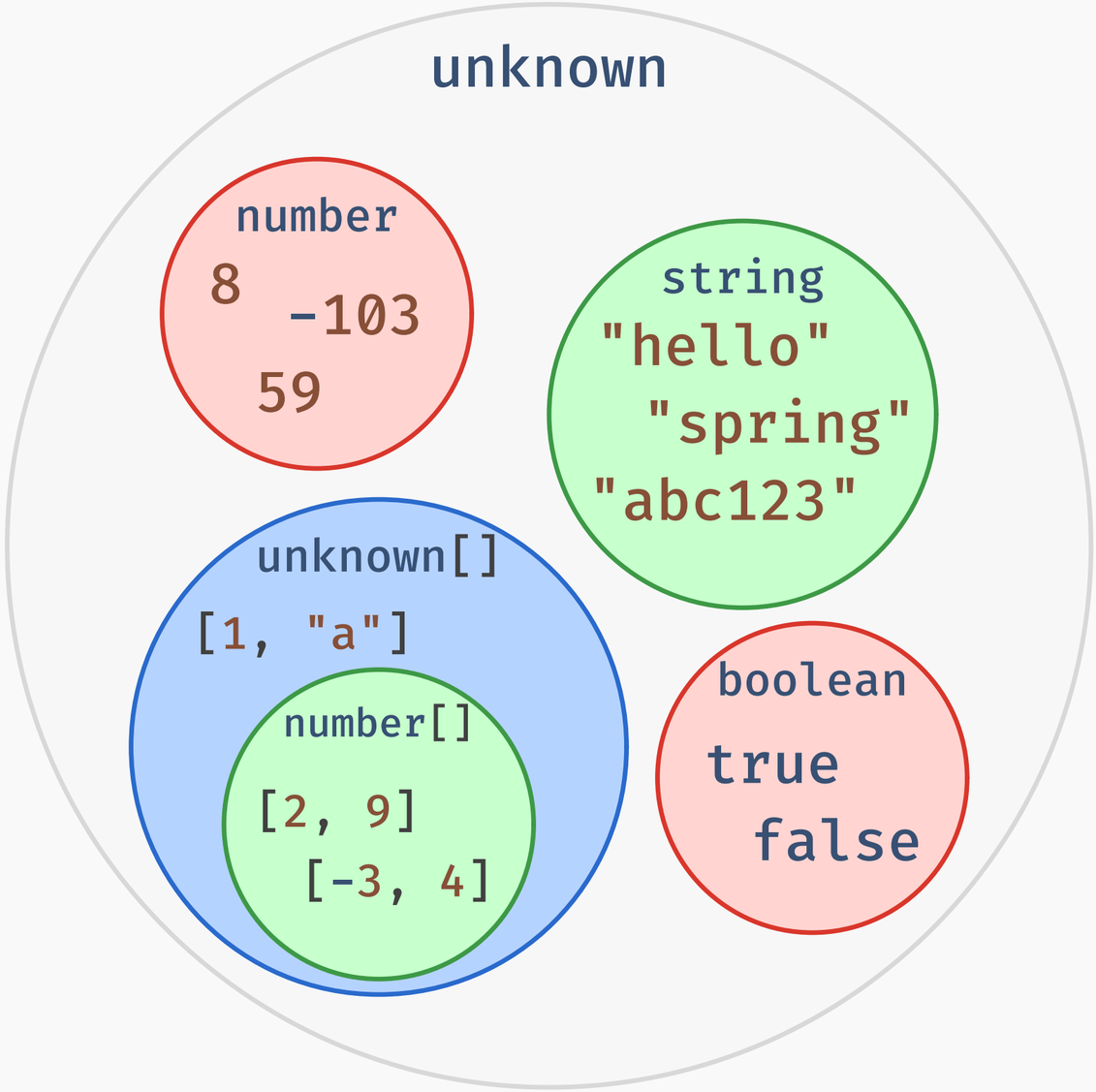
INGREDIENTS

- → Some form of product types (e.g. tuples, structs, records).
- → Some form of sum types (e.g. ADTs/Rust-style enums, union types, sealed interfaces).
- → Exhaustive pattern-matching on sum type variants.

That's it!

A SHIFT IN PERSPECTIVE

What are types? Possible answer: *restrictions*.



But other languages work differently!

Java:
```java
record Point(double x, double y)
```

Rust:
```rust
struct Point { x: f64, y: f64 }
```

Scala:
```scala
case class Point(x: Double, y: Double)
```

Haskell:
```haskell
data Point = Point
  { x :: Double, y :: Double }
```

We can extend the set of possible values!

KEY IDEA 1

Think positive space, not negative space.

```
type Natural = Int where >= 0
```

…, ~~-3~~, ~~-2~~, ~~-1~~, 0, 1, 2, 3, …

KEY IDEA 1

Think positive space, not negative space.

```
type Natural = UInt
```

```
..., -3, -2, -1, 0, 1, 2, 3, ...
```

```
type Integer = (Boolean, Natural)
```

It's easier to add than to subtract!

MORE EXAMPLES

```
type List<T> = < from library >
```

```
type NonEmptyList<T> = (T, List<T>)
```

Thought process: a nonempty list always has one element …

…plus zero or more additional elements.

```
type EvenList<T> = List<(T, T)>
```

Thought process: in an `EvenList`, elements always come in pairs.

Is this “cheating”?

KEY IDEA 2

Decouple representation from interpretation.

~~"My data *is* an array, so I must ascribe it an array type."~~

"My data is a sequence of zero or more elements, which can be represented by an array."

"My data is a sequence of one or more elements, which can be represented by a single element, plus an array."

Data does not have a single "privileged" representation!

```
// at least one of email or phone must be set
struct User {
  id: Int,
  email: Option<EmailAddress>,
  phone: Option<PhoneNumber>
}
```

```
// at least one of email or phone must be set
struct User {
  id: Int,
  email: Option<EmailAddress>,
  phone: if (email == None) PhoneNumber
         else Option<PhoneNumber>
}
```

```
// at least one of email or phone must be set
struct User {
  id: Int,
  contact: UserContact
}


enum UserContact {
  Email(EmailAddress),
  Phone(PhoneNumber),
  Both(EmailAddress, PhoneNumber)
}
```

```
// at least one of email or phone must be set
struct User {
  id: Int,
  contact: Ior<EmailAddress, PhoneNumber>
}


enum Ior<A, B> {
  Left(A),
  Right(A),
  Both(A, B)
}
```

```
// contact is None iff isSystemUser is true
struct User {
  id: Int,
  contact: Option<UserContact>,
  isSystemUser: Boolean
}

enum UserContact {
  Email(EmailAddress),
  Phone(PhoneNumber),
  Both(EmailAddress, PhoneNumber)
}
```

```
// contact is None iff isSystemUser is true
struct User {
  id: Int,
  contact: UserContact
}


enum UserContact {
  System,
  Email(EmailAddress),
  Phone(PhoneNumber),
  Both(EmailAddress, PhoneNumber)
}
```

Constructive modeling can be tricky!

```
struct TimeRange
  { start: Instant, end: Instant }
  where start <= end
```



```
struct TimeRange
  { start: Instant, duration: Duration }
```

“How do I know which representation to pick?”

What is the type system *for*?

(At least as far as correctness goes.)

- → Preventing you from writing `1 + stringVariable`?
- → Formally proving your program correct?
- → Playing taxonomist in your product domain?
- → Writing maximally-precise specifications of the shape of your data at runtime?

My answer: keeping track of the cases to handle!

```
enum UserType { Standard, Admin, System }
```





```
def authenticateUser(request) =
  …
  if (hasSuperuserPrivileges)
    User(type = Admin)
  else
    User(type = Standard)

def processJob(job) =
  let user = User(type = System)
  …
```

```
def authorizeAction(user) =
  match user {
    case Standard => …
    case Admin    => …
    case System   => …
  }
```

Might be very far apart!

```
enum UserType { Standard, Admin, System, Api }
```





```
def authenticateUser(request) =
  …
  if (hasSuperuserPrivileges)
    User(type = Admin)
  else
    User(type = Standard)

def processJob(job) =
  let user = User(type = System)
  …
```

```
def authorizeAction(user) =
  match user {
    case Standard => …
    case Admin    => …
    case System   => …
  }
```



error: non-exhaustive pattern match

Might be very far apart!

A type system is an *obligation propagation machine*.

Use sites inform what the obligations must be!

```
def calculateTotal(entries: List<InventoryLogEntry>) =
  entries.map(entry => entry.change).sum()
```

No need to get fancy: `List` is fine!

```
def getLastChanged(entries: List<InventoryLogEntry>) =
 match entries.head {
   Some(latestEntry) => latestEntry.timestamp
   None              => panic("shouldn't happen")
 }
```

```
def getLastChanged(
      entries: NonEmptyList<InventoryLogEntry>) =
  entries.head.timestamp
```

KEY IDEA 3

Pick the simplest representation that allows writing

```
panic("this shouldn't happen")
```

as little as possible.

In other words: write total functions!

OPTION TYPES

Rust: `enum Option<T> { None, Some(T) }`

Haskell: `data Maybe a = Nothing | Just a`

TypeScript:
```
type Option<T> = T | null
type Option<T> = { some: T } | null
```

Which is more restrictive?

`T` *or* `Option<T>`?

It depends!

```
notifyAboutFailure(user: Option<User>): void
```

```
        def myCoolApiEndpoint() =
          try { … }
          catch { notifyAboutFailure(None) }
```

```
def notifyAboutFailure(user: Option<User>) =
  match user {
    case Some(user) => sendFailureEmail(user.email)
    case None       => ???
  }
```



The Golden Sir
@screaminbutcalm

Me sowing: Haha fuck yeah!!! Yes!!

Me reaping: Well this fucking sucks. What the fuck.

4:14 PM · Mar 12, 2019

201 44K 125K 12K

Let's try again.

```
def notifyAboutFailure(user: User) =
  sendFailureEmail(user.email)

def myCoolApiEndpoint() =
  try { … }
  catch { notifyAboutFailure(???) }
```

Types allow *moving obligations around*.

RECAP

- → Define your data's positive space, not its negative space.
- → Decouple data representation from interpretation.
- → Use type definitions to encode the obligations necessary to write total functions.
- → Push obligations to the places best equipped to handle them.

"As simple as possible, but no simpler."

CONVENIENCE

Fancy type system features are not bad!

```
type NonEmptyList<T> = [T, ...T[]]
```

Take advantage of what your language has to offer!

Just don't get tricked into thinking you *need* them.

THANKS!

- → Define your data's positive space, not its negative space.
- → Decouple data representation from interpretation.
- → Use type definitions to encode the obligations necessary to write total functions.
- → Push obligations to the places best equipped to handle them.

blog: https://lexi-lambda.github.io/
twitter: @lexi_lambda
bluesky: @lexi-lambda.bsky.social